DASAR PEMROGRAMAN

Input, Output, Variabel, dan Tipe Data

Menyimpan dan Mengolah Data dalam Program

Pertemuan 2

TIM DOSEN

Program Studi S1 Informatika | Universitas Pembangunan Nasional Veteran Jakarta

Capaian Pembelajaran Pertemuan 2

Input, Output, Variabel, dan Tipe Data

Setelah mengikuti pertemuan ini, mahasiswa diharapkan mampu:

01

Menjelaskan konsep data, variabel, identifier, dan tipe data dalam bahasa C.

02

Menggunakan fungsi input dan output sederhana, seperti printf() dan scanf().

03

Mendeklarasikan, menginisialisasi, dan menggunakan variabel dengan benar.

04



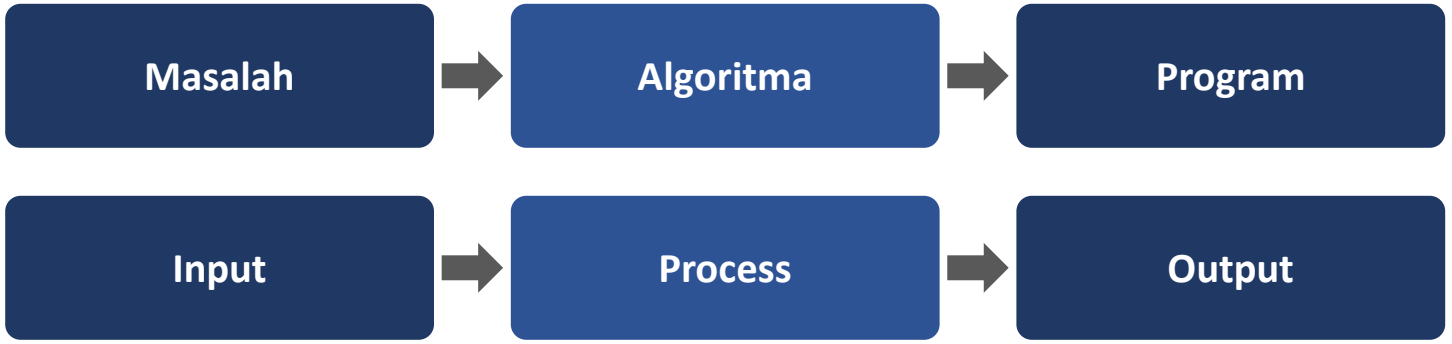
Membedakan serta memilih tipe data dasar C sesuai kebutuhan program.

05

Melakukan tracing sederhana untuk memahami perubahan nilai variabel dalam program.

Review Pertemuan 1

Pada pertemuan sebelumnya, mahasiswa telah mempelajari:



Contoh: Luas Persegi Panjang

```
Input: panjang, lebar
Process: luas = panjang × lebar
Output: luas
```

Pertanyaan untuk membuka Pertemuan 2:

Ketika nilai panjang dan lebar dimasukkan ke dalam program, di mana nilai tersebut disimpan?

JAWABANNYA

Variabel

Tempat di mana nilai panjang dan lebar disimpan selama program berjalan

Data dalam Program

Program bekerja dengan data. Contoh:

`18`	`3.75`	`'A'`	`“Informatika"`	`100000`

Data dapat berasal dari:

- pengguna
- file
- sensor
- sistem lain
- hasil proses sebelumnya

Program menerima → menyimpan → mengolah → menghasilkan output

`80, 75, 85`

↓ Process ↓

`Rata-rata = 80`

Input–Process–Output

Model IPO membantu kita memisahkan tiga hal penting:

Bagian	Pertanyaan
Input	Data apa yang dibutuhkan?
Process	Apa yang dilakukan terhadap data?
Output	Hasil apa yang diinginkan?

Contoh: Menghitung Usia Tahun Depan

Input

`umur`

Process

```
umurTahunDepan
  = umur + 1
```

Output

`umurTahunDepan`

Variabel

Variabel adalah tempat penyimpanan nilai dalam program yang dapat berubah selama program berjalan.



Nama variabel (umur) tetap sama, tetapi nilainya dapat berubah dari waktu ke waktu.

Variabel Memiliki Nama dan Nilai

```
nilaiUjian = 85;
```

Identifier

nilaiUjian — nama yang digunakan untuk mengenali variabel.

Value

85 — nilai yang sedang disimpan oleh variabel tersebut.

Variabel juga memiliki Data Type (dibahas setelah ini).

Identifier

Identifier adalah nama yang digunakan untuk variabel, konstanta, atau fungsi.

Nama	Status	Alasan
`umur`	Valid	Diawali huruf
`_nilai`	Valid	Diawali underscore
`nilai2`	Valid	Angka tidak di awal
`2nilai`	Tidak valid	Diawali angka
`nilai ujian`	Tidak valid	Mengandung spasi
`int`	Tidak valid	Kata kunci (keyword) bahasa C

Catatan: bahasa C bersifat case-sensitive — nilai dan Nilai adalah dua identifier yang berbeda.

Naming yang Baik

Nama variabel sebaiknya menjelaskan maksud data yang disimpan.

Kurang Jelas

```
x = 10;
y = 8;
```

Lebih Jelas

```
jumlahMahasiswa = 10;
jumlahLulus = 8;
```

Catatan: nama singkat seperti i, x, atau n masih wajar digunakan apabila konteksnya sudah jelas, misalnya sebagai variabel penghitung pada perulangan (dibahas pada pertemuan mendatang).

Konstanta

Konstanta adalah nilai yang tidak berubah selama program berjalan.

```
PI = 3.14159
```

Aspek	Variabel	Konstanta
Nilai	Dapat berubah	Tetap
Contoh	umur = 18; → 19;	PI = 3.14159 (tetap)

Catatan: penulisan konstanta menggunakan const atau #define akan dibahas lebih lanjut pada pertemuan mendatang.

Mengapa Kita Membutuhkan Tipe Data?

Perhatikan kembali data-data berikut:

`18`	`3.75`	`'A'`
bilangan bulat	*bilangan pecahan*	*karakter*

Ketiganya berbeda jenis — inilah yang disebut tipe data.

Tipe data menentukan jenis nilai yang dapat disimpan dalam suatu variabel.

Tipe Data Dasar dalam C

Tipe Data	Jml Bit	Digunakan untuk	Rentang Data Umum	Format I/O	Contoh	Ket.
`int`	32 bit	Bilangan bulat	-2.147.483.648 s.d. 2.147.483.647	printf: %d scanf: %d	18	Tanpa bagian desimal
`float`	32 bit	Bilangan pecahan	$\pm 1{,}2 \times 10^{-38}$ s.d. $\pm 3{,}4 \times 10^{38}$	printf: %f scanf: %f	3.75f	Presisi sekitar 6–7 digit
`double`	64 bit	Bilangan pecahan presisi lebih tinggi	$\pm 2{,}2 \times 10^{-308}$ s.d. $\pm 1{,}8 \times 10^{308}$	printf: %f scanf: %lf	3.75	Presisi sekitar 15–16 digit
`char`	8 bit	Satu karakter	1 byte	printf: %c scanf: %c	'A'	Karakter tunggal
`char[]` (string dasar)	Bergantung kapasitas	Rangkaian karakter	Bergantung ukuran array	printf: %s scanf: %s	"Informatika"	String di C berupa array karakter

Catatan: ukuran tipe data dapat bergantung pada compiler dan arsitektur sistem. Nilai pada tabel merupakan ukuran dan rentang yang umum digunakan. String diperkenalkan secara ringan pada pertemuan ini dan akan dibahas lebih mendalam pada Pertemuan 11.

Tipe Data int

int digunakan untuk menyimpan bilangan bulat, tanpa bagian pecahan.

```
int umur = 18;
```

Cocok digunakan untuk:

- **umur**
- **jumlah barang**
- **jumlah mahasiswa**

float dan double

float dan double digunakan untuk menyimpan bilangan pecahan (desimal).

float

```
float berat = 55.5;
```

double

```
double rataRata = 79.66;
```

Perbedaan: double dapat menyimpan bilangan pecahan dengan presisi (tingkat ketelitian) lebih tinggi daripada float.

Tipe Data char

char digunakan untuk menyimpan satu karakter tunggal.

```
char grade = 'A';
```

'A'

Karakter tunggal → tipe char
Ditulis dengan petik tunggal

"A"

Rangkaian karakter (string) → bukan char
Ditulis dengan petik ganda

Memilih Tipe Data

Diskusikan tipe data yang sesuai untuk setiap data berikut:

Data	Tipe Data yang Sesuai
Jumlah siswa	int
Tinggi badan	float / double
Huruf grade	char
Harga barang	int / double
Nama pengguna	string (dasar)

Pesan Utama: pemilihan tipe data yang tepat membuat program lebih efisien dan mudah dipahami.

Deklarasi dan Inisialisasi Variabel

Deklarasi

Memberi tahu compiler nama dan tipe data variabel yang akan digunakan.

```
int umur;
```

Inisialisasi

Memberikan nilai awal pada variabel saat dideklarasikan.

```
int umur = 18;
```

int Data Type **umur** Identifier **18** Value

Assignment

Assignment adalah proses memberikan atau mengganti nilai suatu variabel menggunakan tanda =.

```
int umur = 18;

umur = 19;   // mengganti nilai
```

Nilai 18 digantikan oleh nilai 19. Variabel umur tetap sama, hanya isinya yang berubah.

Assignment Bukan Persamaan Matematika

Penting: tanda = pada bahasa C berarti “berikan nilai”, bukan “sama dengan” seperti pada matematika.

```
x = x + 1;
```

1. Ambil nilai x saat ini
2. Tambahkan 1
3. Simpan kembali hasilnya ke x

Jika x = 5, maka setelah x = x + 1; nilai x menjadi 6.

Input

Input adalah proses menerima data dari luar program, misalnya dari pengguna melalui keyboard.

Fungsi scanf()

```
int umur;
scanf("%d", &umur);
```

Tanda & (address-of) akan dijelaskan lebih rinci pada Pertemuan 11 (Pointer).

Output

Output adalah proses menampilkan hasil dari program, misalnya ke layar.

Fungsi printf()

```
printf("Umur: %d", umur);
```

Keluaran: Umur: 18

Format Specifier Dasar

Format specifier memberi tahu scanf()/printf() jenis data yang sedang diproses.

Tipe Data	Format Specifier
`int`	`%d`
`float`	`%f`
`double`	`%lf (scanf) / %f (printf)`
`char`	`%c`

Escape Sequence Dasar

Escape sequence digunakan untuk karakter khusus dalam teks output.

\n

Pindah ke baris baru

```
printf("Halo\n");
```

\t

Memberi tabulasi (indentasi)

```
printf("A\tB");
```

STUDI KASUS 1

Menghitung Luas Persegi Panjang

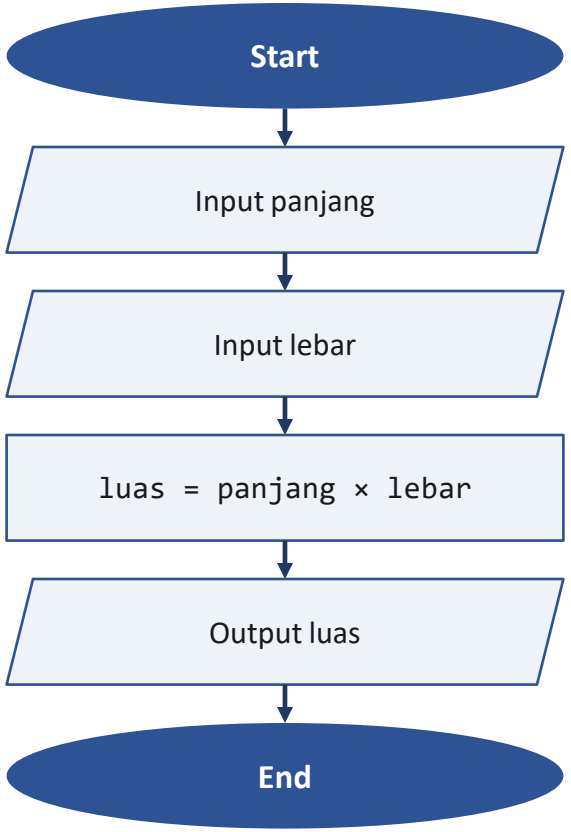
Buat program untuk menghitung luas persegi panjang berdasarkan panjang dan lebar yang dimasukkan pengguna.

Input	Process	Output
panjang, lebar	`luas = panjang × lebar`	luas

```
panjang = 10          lebar = 5
```

Case 1 — Flowchart

Menghitung Luas Persegi Panjang



Contoh Nilai

```
panjang = 10
lebar = 5
luas = 10 × 5 = 50
```

Case 1 — Pseudocode dan Kode C

Pseudocode

```
START

INPUT panjang
INPUT lebar

luas ← panjang × lebar

OUTPUT luas

END
```

Kode C

```c
#include <stdio.h>

int main() {
  int panjang, lebar, luas;

  printf("Panjang: ");
  scanf("%d", &panjang);
  printf("Lebar: ");
  scanf("%d", &lebar);

  luas = panjang * lebar;

  printf("Luas = %d\n", luas);
  return 0;
}
```

Case 1 — Tracing dan Output

Langkah	panjang	lebar	luas
Awal	–	–	–
Input panjang	10	–	–
Input lebar	10	5	–
luas = panjang × lebar	10	5	50
Output	10	5	50

Output Program

```
Masukkan panjang: 10
Masukkan lebar: 5
Luas = 50
```

STUDI KASUS 2

Menghitung Total Belanja

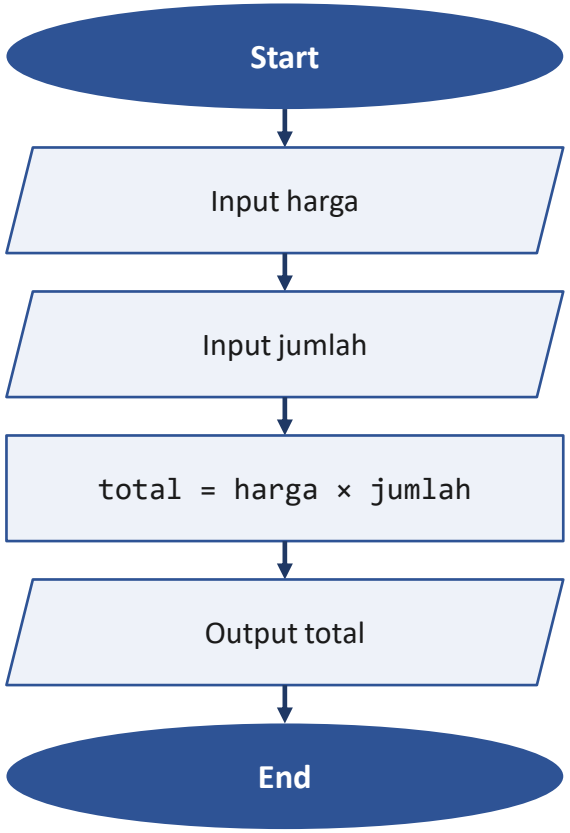
Seorang pelanggan membeli beberapa barang dengan harga yang sama. Program menghitung total pembayaran berdasarkan harga satu barang dan jumlah barang.

Input	Process	Output
harga, jumlah	`total = harga × jumlah`	total pembayaran

```
harga = 25000          jumlah = 4
```

Case 2 — Flowchart

Menghitung Total Belanja



Contoh Nilai

```
harga = 25000
jumlah = 4
total = 25000 × 4
      = 100000
```

Case 2 — Pseudocode dan Kode C

Pseudocode

```
START

INPUT harga
INPUT jumlah

total ← harga × jumlah

OUTPUT total

END
```

Kode C

```c
#include <stdio.h>

int main() {
  int harga, jumlah, total;

  printf("Harga barang: ");
  scanf("%d", &harga);
  printf("Jumlah barang: ");
  scanf("%d", &jumlah);

  total = harga * jumlah;

  printf("Total = %d\n", total);
  return 0;
}
```

Case 2 — Tracing dan Output

Langkah	harga	jumlah	total
Input harga	25000	–	–
Input jumlah	25000	4	–
Hitung total	25000	4	100000
Output	25000	4	100000

Output Program

```
Harga barang: 25000
Jumlah barang: 4
Total pembayaran = 100000
```

STUDI KASUS 3

Menghitung Rata-Rata Tiga Nilai

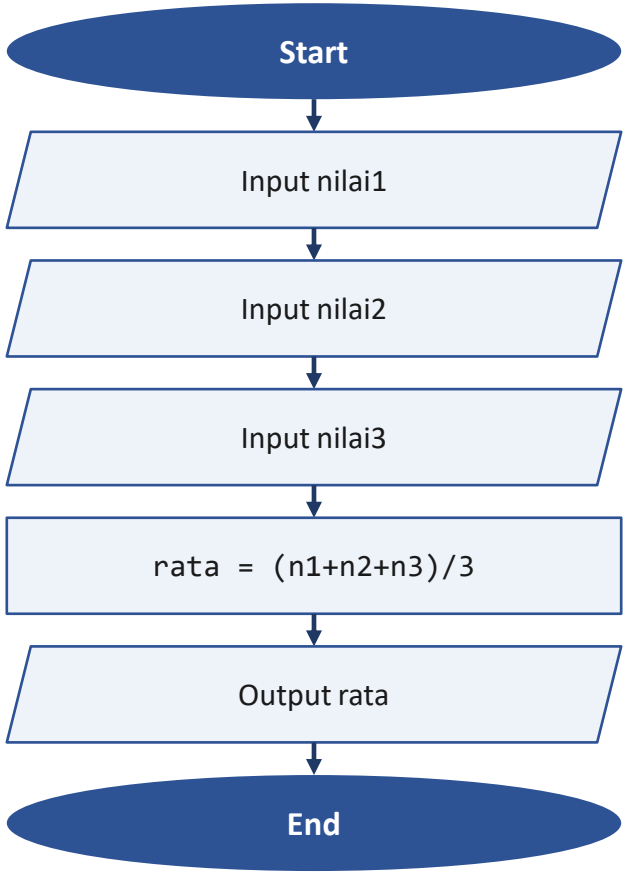
Hitung rata-rata dari tiga nilai mahasiswa. Case ini penting untuk memperkenalkan data pecahan.

Input	Process	Output
nilai1, nilai2, nilai3	`(nilai1+nilai2+nilai3) / 3`	rata-rata

```
80 75 85
```

Case 3 — Flowchart

Menghitung Rata-Rata Tiga Nilai



Contoh Nilai

```
nilai1 = 80
nilai2 = 75
nilai3 = 85
rata = (80+75+85)/3
       = 80
```

Case 3 — Pseudocode dan Kode C

Pseudocode

```
START

INPUT nilai1
INPUT nilai2
INPUT nilai3

rata ← (nilai1+nilai2
         +nilai3) / 3

OUTPUT rata

END
```

Kode C (menjaga hasil pecahan)

```c
#include <stdio.h>

int main() {
  float n1, n2, n3, rata;

  printf("Nilai 1: ");
  scanf("%f", &n1);
  printf("Nilai 2: ");
  scanf("%f", &n2);
  printf("Nilai 3: ");
  scanf("%f", &n3);

  rata = (n1+n2+n3) / 3;

  printf("Rata = %.2f\n", rata);
  return 0;
}
```

Case 3 — Tracing dan Output

Langkah	nilai1	nilai2	nilai3	rata
Input 1	80	–	–	–
Input 2	80	75	–	–
Input 3	80	75	85	–
Hitung	80	75	85	80
Output	80	75	85	80

Output Program

```
Nilai 1: 80
Nilai 2: 75
Nilai 3: 85
Rata-rata = 80.00
```

Mengapa Case 3 Menggunakan float?

Bandingkan dua kondisi berikut:

```
Data: 80, 75, 85

Rata-rata = 80
```

Hasil bulat

```
Data: 80, 75, 84

Rata-rata = 79.67
```

Hasil pecahan!

Pesan Utama: pemilihan tipe data ditentukan oleh kemungkinan nilai yang harus direpresentasikan, bukan hanya oleh contoh data yang kebetulan bulat.

Tracing Perubahan Nilai

Selain studi kasus, mahasiswa perlu latihan tracing murni terhadap perubahan nilai variabel.

Kode

```
x ← 5
y ← 2
z ← x
x ← 10
y ← x + z
```

Langkah	x	y	z
x ← 5	5	–	–
y ← 2	5	2	–
z ← x	5	2	5
x ← 10	10	2	5
y ← x + z	10	15	5

Hasil akhir: x = 10, y = 15, z = 5

Penting: perubahan x menjadi 10 tidak mengubah z, karena nilai 5 sudah disalin ke z sebelumnya. Ini fondasi penting sebelum mempelajari pointer.

Kesalahan Umum

1. Salah Pilih Tipe Data

int ipk = 3.75; — tipe tidak sesuai karakteristik data (bagian pecahan hilang).

2. Variabel Tanpa Nilai Jelas

int nilai; printf("%d", nilai); — menggunakan variabel yang belum diberi nilai.

3. Salah Menulis Identifier

int 1nilai; — identifier tidak boleh diawali angka.

4. Salah Memahami Assignment

x = x + 1; — dianggap persamaan matematika, padahal assignment.

5. Format Tidak Sesuai Tipe

Memperlakukan integer dan floating-point dengan format specifier yang sama.

Pengenalan Integer Division

PENGANTAR MENUJU PERTEMUAN 3

```
int a = 5;
int b = 2;

printf("%d", a / b);
```

Hasil Sebenarnya

2

Bukan

2.5

Mengapa? Karena kedua operand bertipe integer. Pembahasan operator, ekspresi, precedence, dan type conversion akan dibahas pada Pertemuan 3.

LATIHAN KELAS

A. Pilih Tipe Data

Data	Tipe?
Jumlah mahasiswa	?
IPK	?
Nilai huruf	?
Tinggi badan	?
Jumlah kendaraan	?

B. Identifier: Valid atau Tidak?

```
nilai1
```

```
1nilai
```

```
nilai_akhir
```

```
nilai akhir
```

```
totalHarga
```

```
int
```

LATIHAN KELAS

C. Case Mini — Menghitung Gaji

Buat algoritma untuk menghitung gaji berdasarkan jumlah jam kerja dan upah per jam.

Mahasiswa diminta membuat:



Implementasi kode C dapat dijadikan latihan praktikum atau tugas lanjutan.

Concept Check

Beberapa pertanyaan singkat untuk mengukur pemahaman:

1. Apa perbedaan variabel dan konstanta?
2. Mengapa umur biasanya lebih cocok menggunakan int daripada float?
3. Apa perbedaan deklarasi dan inisialisasi?
4. Apa arti dari x = x + 1;?
5. Mengapa pemilihan tipe data penting?

Rangkuman Pertemuan 2

DATA → VARIABLE → IDENTIFIER → DATA TYPE → INPUT → PROCESS → ASSIGNMENT → OUTPUT → TRACING

Tiga studi kasus lengkap telah dibahas melalui alur berikut:

CASE → IPO → FLOWCHART → PSEUDOCODE → C CODE → TRACING → OUTPUT

01 Case 1 — Luas Persegi Panjang

Fokus: int, input/output, variabel.

02 Case 2 — Total Belanja

Fokus: representasi data dan perubahan nilai.

03 Case 3 — Rata-Rata Nilai

Fokus: float, pemilihan tipe data, hasil pecahan.